Canary vs Veil

A code-level comparison for the ETHGlobal Mumbai “Best Use of Anon-RPC” bounty. Both repositories were cloned and read on 7 November 2026: SCARPxVeNOM/canary_repo and SCARPxVeNOM/veil.

Verdict

Canary is the stronger Anon-RPC project. It sends more of a dapp's traffic through Anon-RPC, it is fast enough to use in a real session, it does not depend on the Ethereum Foundation (EF) demo gateway that went down during the hackathon, and it improves Anon-RPC itself with two tor-js patches. Veil is a careful, focused extension with a well-designed fallback policy, but it is a much smaller piece of engineering with real gaps in coverage and isolation. Veil won 1st place because its pitch was clearer, not because its code was better.

Scope of this review

- I read each project's interception layer, routing logic and Anon-RPC worker integration, and cite files below.
- I ran Canary's RPC pipeline unit tests (`extension/test/pipeline.test.mjs`): 13 passed, 0 failed.
- I did not run either extension end to end against live dapps or the gateways. Latency figures are each team's own published numbers.

1. Scoring against the bounty's judging criteria

Criterion	Canary	Veil	Edge
Works end to end	Runs its own public Tor gateways (Dockerfile in `gateway/`), with EF's gateway as a fallback. Tor startup is about 4 s.	The default is EF's demo gateway (`src/background.ts:9-10`), which stopped completing connections on 6 Nov. The gateway can be changed in settings, but Veil runs none of its own.	Canary
Demo user experience	Batching, caching, deduplication, pre-warmed circuits and 3 s hedged reads. A warm read takes about 0.85 s, and 20 reads take about 2 s.	Each read is a separate `worker.fetch` with no batching or caching. That is about 1 s or more per read, and Veil publishes no latency figures.	Canary
Share of traffic through Anon-RPC	Hooks fetch, XHR, WebSocket and sendBeacon, plus browser blocking rules (declarativeNetRequest) for pixels. 237 of 237 RPC calls on 1inch went through Anon-RPC.	Hooks fetch and XHR only. WebSocket RPC bypasses it completely.	Canary
Shippable after the hackathon	Fails closed, allows direct access only per site, and publishes its agent packages to npm. It has more moving parts to maintain.	Small codebase with a clean policy for falling back. Without batching it is slow on real dashboards.	Slight Canary
Technical depth	Two tor-js patches (about 1,370 diff lines) with benchmarks, a dead-gateway recovery fix, reproducible worker builds and its own gateway infrastructure.	Per-account routing heuristic, JSON-RPC id blinding and a re-route classifier.	Canary, clearly

2. Codebase at a glance

	Canary	Veil
TS/JS source (approx.)	About 17,500 lines	About 3,200 lines (excluding experiments)
Commits / authors	50 commits, 3 authors, from 5 Nov	78 commits, 1 author, all dated 6 Nov

	Canary	Veil
Components	Extension, RPC pipeline, gateway Docker image, tor-js patches, ENS packages, scanner, swap app, agent gateway and SDK (published to npm), dashboard	Extension, local helper, Tauri desktop app, marketing site
Automated tests	Unit tests (pipeline, ENS), e2e, wallet-hold, live stress scripts	Playwright e2e scripts (e2e, fallback, policy, options, helper)

3. Where Canary is technically ahead

3.1 Coverage: more channels intercepted

Canary's MAIN-world script wraps `fetch`, `XMLHttpRequest`, `WebSocket.prototype.send` and `navigator.sendBeacon` (`extension/src/shim.ts:29-34`). Pixels and script tags are caught by browser-level blocking rules. Veil's `src/inject.ts` wraps only fetch and XHR. A dapp that reads over `wss://`, which is common for subscriptions and some viem setups, sends those reads directly, with the user's IP, and Veil neither routes nor reports them.

3.2 Isolation: addresses come from the wallet, not from guesswork

Veil learns which addresses are yours only from `eth_getBalance` and `eth_getTransactionCount` calls (`src/shared/routing.ts`, `learnableAddresses`). Until a dapp makes one of those calls, reads such as `balanceOf(you)` and allowances go out on the shared “neutral” circuit. Several accounts can then share an exit, which undermines Veil's main promise of one circuit per account.

Canary reads the address directly from the wallet: it watches `eth_accounts`, `eth_requestAccounts` and `accountsChanged`, and silently asks for `eth_accounts` at load (`extension/src/shim.ts:128-140`). When the user switches accounts, it holds back the wallet's account announcement until the blocking rule for the new address is installed (`extension/test/wallet-hold.mjs`).

3.3 Performance: built for Tor's latency

- **Batching:** calls for the same endpoint *and* isolation key that arrive within 15 ms leave as one JSON-RPC batch. Batches never mix keys, so two addresses never share a circuit (`extension/src/rpc-pipeline.ts`). 20 reads went from 43-55 s to 1.6-2.1 s.
- **Caching and deduplication:** chain IDs and contract code are cached for good, and the block number for 2 s. On app.uniswap.org, 20 of 24 calls were exact duplicates.
- **Batch splitting:** if an endpoint refuses a batch for its size (HTTP 400 or 413), Canary retries it in halves and remembers that endpoint's limit.
- **Hedged reads:** a read with no answer after 3 s is re-sent on a new circuit derived from the same key. If the hedge wins, that circuit becomes the key's circuit from then on (`extension/src/background.ts:419-431`). Recovered reads dropped from about 4.2 s to about 0.8 s.
- **Pre-warmed circuits:** a circuit is built as soon as the address is known, before the dapp's first read.
- **Bundled Tor directory snapshot:** saves 11-13 s on first startup.

Veil sends every page request as its own `worker.fetch`. Under the cost Canary measured for tor-js, a dashboard that fires 20 reads waits tens of seconds. That is the difference between a demo and something people can use.

3.4 Improving Anon-RPC itself

`tor-js-patches/` contains two patches to EF's tor-js, about 1,370 diff lines in total:

- **HTTP keep-alive per circuit:** a warm read went from 827 ms to 234 ms, and 20 parallel reads from 2.0 s to 0.98 s. Connections are never reused across isolation keys.

- **Gateway liveness check:** a gateway that goes quiet is dropped after two missed checks. Recovery time fell from 17-31 s to 10.5-14 s, and each failure costs one failed read instead of two.
- **Reproducible builds:** an unpatched build produces exactly the hash of the worker currently pinned on-chain. That is the route for getting these patches accepted upstream.

This is the “improve support for an existing harness” direction the bounty explicitly invites. Veil uses `@anon-rpc/browser-extension-harness` as it ships and makes no changes to it.

3.5 Reliability: its own gateways

EF's demo gateway took 18-43 s to start Tor and stopped completing connections on 6 November. Canary runs its own gateways in Docker behind a UDP tunnel, starting in about 4 s, and keeps EF's gateway as a fallback. Veil's default is EF's demo gateway, so on judging day its out-of-the-box path was the one that had broken.

3.6 Correct chain for every endpoint

Canary works out each endpoint's chain from a list of known providers or an `eth_chainId` probe over Tor, and handles several chains per page. Veil re-routes only for a hardcoded list of single-chain hosts. That is safe but narrow: anything outside the list ends up at the per-site prompt.

4. Where Veil is better or equal (being fair)

Point	Assessment
Fallback policy	Veil tries the dapp's own endpoint over Tor first, re-routes only after a real block and only to a known single-chain provider, and otherwise shows a per-site ask/block/allow dialog. It is clean and easy to explain. **Edge: Veil.**
JSON-RPC id blinding	Veil randomises ids. Canary rebuilds every batch with ids 0..n (`rpc-pipeline.ts:122`), so viem's global counter never reaches the provider either. **Even**, but only Veil says so in its write-up.
Honesty about limits	Veil states clearly what providers still see and names Private Information Retrieval as future work. **Edge: Veil** on presentation.
Focus	Veil is entirely about Anon-RPC. Canary's ENS, Uniswap and agent work dilutes its pitch for this bounty. **Edge: Veil** on pitch, not on substance.
Exit-IP visibility	Veil shows each circuit's exit IP in its UI by querying an IP echo service over that circuit. A nice touch for a demo.

5. Veil weaknesses a judge could check in the code

Issue	Evidence	Impact
No WebSocket interception	`src/inject.ts` wraps only fetch and XHR	wss:// RPC reads leak the user's IP
Late address discovery	`routing.ts` learns addresses only from getBalance and getTransactionCount	Early per-account reads share the neutral circuit
No batching or caching	One `worker.fetch` per request in `routeRpc`	Pages that make many reads feel slow over Tor
Depends on EF demo gateway	Default gateway setting in `background.ts`	Fragile: that gateway failed on 6 Nov
No upstream contribution	Harness used as shipped	Less technical depth

6. Why Canary did not win, and how to fix it

- **Diluted pitch.** Anon-RPC is one of six pillars in Canary's write-up. Judges for a single bounty look for one sharp story.

- **Too many numbers, too little narrative.** Benchmarks without a clear before-and-after demo are hard to check in a 3-minute judging slot.
- **Silence on privacy subtleties.** Canary already avoids leaking viem's id counter but never says so. Veil gets credit for mentioning it.
- **Swapping the provider by default** (rather than re-routing only after a block) needs a one-line justification: the dapp's provider URL often carries an API key that identifies the dapp.

Suggested Anon-RPC-only pitch

> “Canary sends **every** dapp read through Anon-RPC, over fetch, XHR and WebSocket, and makes it fast enough to use: 20 reads in 2 s instead of 50 s. We found and fixed two tor-js bottlenecks (warm read 827 ms to 234 ms), kept it running when EF's gateway died by deploying our own, and our builds reproduce the on-chain worker hash so the patches can go upstream. Each address gets its own circuit, taken from the wallet rather than guessed, and if Tor fails we fail closed.”

7. Conclusion

On the bounty's stated criteria (end to end, user experience, share of traffic, shippability, technical depth), Canary wins four outright and edges the fifth. Veil's advantages are presentation, focus and one well-designed fallback policy. Canary is the better piece of engineering for Anon-RPC. Veil was the better-told story.